

CASE STUDY REPORT

Design and Deployment of a Highly Available, Multi-Tier Cloud-Native Application using the AWS Well-Architected Framework

Project	TaskFlow – Highly Available AWS Application
Student	Atharva Singh
Roll Number	202401100100072
AWS Region	Asia Pacific (Mumbai) – ap-south-1
Application	SOC Analyst Agent Web Application

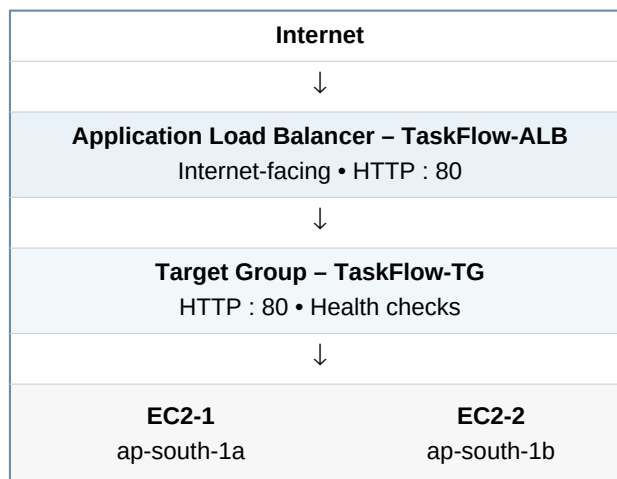
1. Abstract

This case study presents the design and deployment of ~~SOC Analyst Agent~~ ~~TaskFlow~~ ~~Highly Available~~ ~~Application~~ ~~hosted on~~ Amazon Web Services (AWS). The solution uses a multi-tier architecture with an internet-facing Application Load Balancer (ALB), a target group, and two Amazon EC2 application servers distributed across two Availability Zones. The design improves availability, fault tolerance, traffic distribution, and operational visibility while following core principles of the AWS Well-Architected Framework.

2. Objectives

- Deploy the TaskFlow application on AWS using EC2-based compute resources.
- Provide high availability by running two application instances in separate Availability Zones.
- Distribute incoming HTTP traffic through an Application Load Balancer.
- Use health checks to route traffic only to healthy targets.
- Apply security-group rules so that the load balancer communicates with the application tier.
- Document the infrastructure, deployment process, testing evidence, and architecture.

3. AWS Architecture



The two EC2 instances are registered as healthy targets. If one instance becomes unhealthy, the ALB can continue forwarding requests to the remaining healthy instance.

4. Infrastructure and AWS Services

Component	Configuration / Purpose
Amazon VPC	Isolated network for the TaskFlow application.
Public Subnets	Two subnets in different Availability Zones for resilient deployment.
Internet Gateway	Provides internet connectivity for the public-facing architecture.
Application Load Balancer	Receives HTTP requests and distributes them across healthy EC2 targets.
Target Group	TaskFlow-TG; HTTP on port 80 with target health checks.
EC2 Instance 1	TaskFlow-EC2-1; application server in ap-south-1a.
EC2 Instance 2	TaskFlow-EC2-2; application server in ap-south-1b.
Security Groups	Separate ALB and EC2 traffic controls; EC2 HTTP access is restricted to the ALB security group.
Nginx	Web server/reverse-proxy layer used on the EC2 application hosts.

5. Deployment Workflow

- Develop and build the TaskFlow frontend locally.
- Create the AWS networking foundation: VPC, public subnets, routing and Internet Gateway.
- Launch two EC2 instances in separate Availability Zones.
- Install and configure Nginx on the EC2 servers and deploy the application build.
- Configure the EC2 security group for application traffic from the ALB security group.
- Create the target group and register both EC2 instances on HTTP port 80.
- Create the internet-facing Application Load Balancer with an HTTP : 80 listener.
- Forward listener traffic to TaskFlow-TG and verify both targets report Healthy.
- Test the ALB endpoint and perform a failover test by temporarily stopping one EC2 instance.

6. Security Design

Security is implemented using AWS security groups. The ALB security group accepts HTTP traffic for the public application endpoint. The EC2 security group permits application HTTP traffic from the ALB security group rather than exposing the application tier directly to arbitrary sources. SSH access is restricted to the required administrative source. This separation follows the principle of least privilege at the network layer.

7. High Availability and Testing

The deployment uses two EC2 instances located in different Availability Zones and an Application Load Balancer in front of them. The target group health checks provide a mechanism for detecting unhealthy instances. A successful deployment should show both targets as Healthy and the application should remain accessible through the ALB when one backend instance is temporarily unavailable.

Evidence	What it demonstrates
EC2 Instances screenshot	Two running application servers.
Target Group screenshot	Both targets registered and Healthy.
Security Group screenshot	Controlled ALB-to-EC2 traffic.
Load Balancer screenshot	Internet-facing ALB and HTTP listener.
ALB application screenshot	End-to-end application accessibility.
Failover screenshot	Application remains available after one backend is stopped.

8. AWS Well-Architected Framework Alignment

Pillar	Application of the pillar in TaskFlow
Operational Excellence	Documented deployment steps, health checks, monitoring evidence and repeatable infrastructure configuration.
Security	Security groups separate public load-balancer traffic from the EC2 application tier and restrict administrative access.
Reliability	Two EC2 instances in separate Availability Zones, ALB health checks and automatic traffic distribution improve fault tolerance.
Performance Efficiency	ALB distributes requests across available application servers and avoids sending traffic to unhealthy targets.
Cost Optimization	The demonstration uses a small EC2-based deployment appropriate for a student case study; resources can be scaled as needed.
Sustainability	Right-sized compute and load distribution help avoid unnecessary resource usage; the architecture can scale across multiple regions.

9. Infrastructure as Code / Deployment Files

AWS CloudFormation IaC Generator was used to scan the deployed AWS environment and prepare an infrastructure template. The generated CloudFormation template is intended to document the deployed infrastructure and can be included with the submission under **infrastructure/TaskFlow-Infrastructure.yaml**. The source application is maintained separately under the source-code directory.

10. Repository and Submission Structure

YourRollNumber_YourName/
■■■ Case_Study_Report.pdf
■■■ README.md
■■■ Architecture_Diagram.png
■■■ infrastructure/
■ ■■■ TaskFlow-Infrastructure.yaml
■■■ source-code/
■ ■■■ cloudapp/
■■■ screenshots/
■■■ 01_EC2_Instances.png
■■■ 02_VPC_Subnets.png
■■■ 03_Security_Groups.png
■■■ 04_Load_Balancer.png
■■■ 05_Target_Group_Healthy.png
■■■ 06_ALB_Website.png
■■■ 10_Failover_Test.png

11. Conclusion

TaskFlow demonstrates a practical highly available AWS deployment using an Application Load Balancer and two EC2 application servers across separate Availability Zones. The architecture introduces traffic distribution, health-based routing and network-level security while remaining simple enough for a student cloud case study. The deployment evidence, source code, architecture diagram and infrastructure template together provide a reproducible record of the implementation.

GitHub Repository: To be added after the public repository is created.

Note: Replace the Roll Number placeholder and GitHub placeholder before final submission. Add the final architecture image, deployment screenshots and generated CloudFormation YAML to the submission folder.